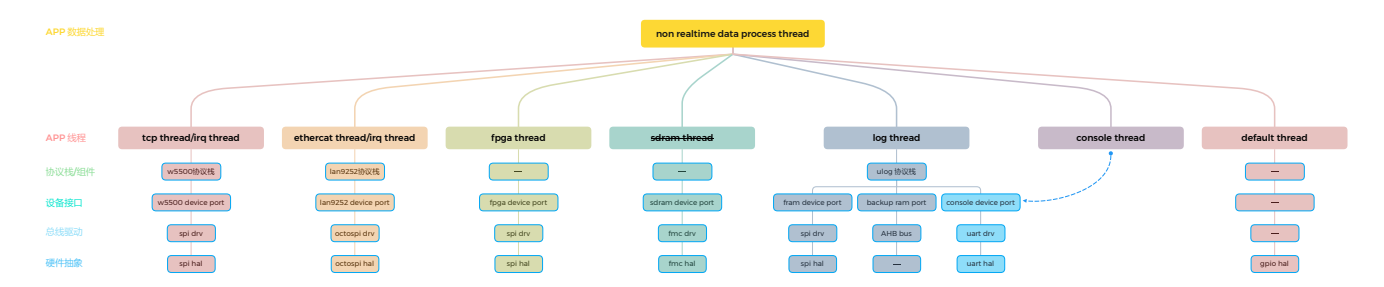


1. 工程文件架构



- 工程文件夹说明
 - application: 存放应用程序代码，按功能划分为ethernet、fpga、tcp等子文件夹
 - board: 存放板级支持代码，如stm32h7xx链接文件、启动文件等
 - components: 存放组件代码，如backtrace、ulog、finsh等组件，以及一些通用基础性文件
 - drivers: 存放驱动代码，按外设类型划分external_drivers和internal_drivers两个文件目录。
 - external_drivers: 存放芯片外部板载设备驱动代码，如w5500、lan9252、sdram等
 - internal_drivers: 存放芯片内部设备驱动代码，如uart、spi、i2c等
 - libraries: 存放库文件
 - CMSIS: 存放ARM CMSIS标准库
 - STM32H7xx_HAL_Driver: 存放STM32H7xx系列HAL库
 - Middlewares: 存放中间件代码，如FreeRTOS、LwIP、FatFs、LwIP等
 - test_case: 存放测试用例

2. 工程修改说明

为便于工程移植和修改，工程架构上设计了一些文件目录和文件，其与STM32CubeMX生成的工程目录结构相似。下面将介绍如何修改这些文件。

2.1 使用STM32CubeMX更新工程配置

1. 为便于解释和理解，下述描述中用到的工程指代说明：

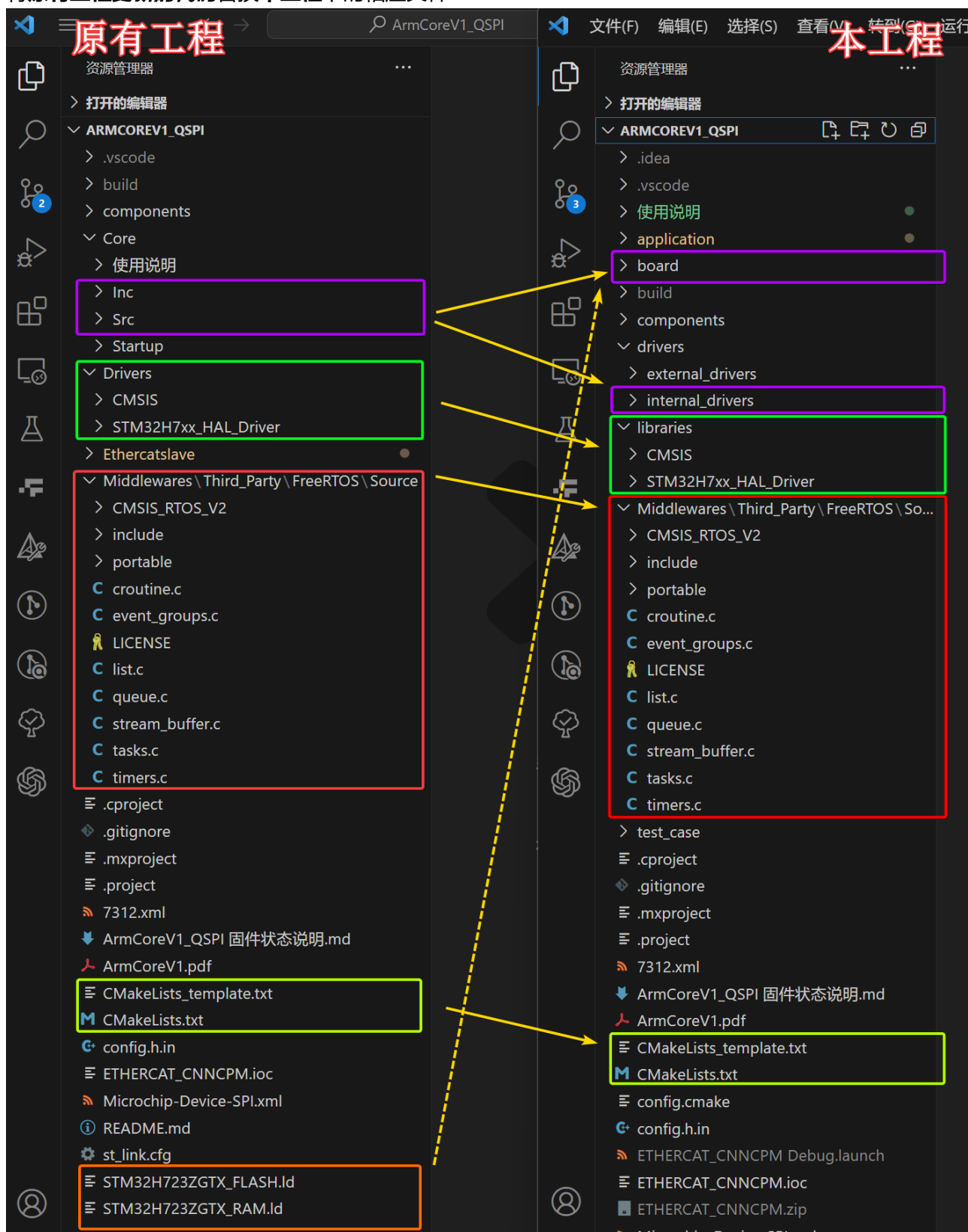
原有工程: 指代使用STM32CubeMX生成的工程，其目录结构与STM32CubeMX生成的目录结构相同

本工程: 指代本项目工程，其目录结构与原有工程不同

1. 修改说明

- 在**原有工程**下打开STM32CubeMX配置文件，根据需要修改外设配置，重新生成项目代码

- 将原有工程更新的代码替换本工程下的相应文件



- Middlewares文件夹内容直接替换覆盖，因backtrace组件涉及到FreeRTOS内核文件更改，需要注意将该部分差异恢复
- Drivers文件夹内容直接替换覆盖本工程下的libraries文件夹内容
- Core文件夹下内容按照片上外设差异替换本工程下的drivers/internal_drivers文件夹内容，若有新增，则新建文件夹xxx存放该外设源文件和头文件
- stm32h7xx_it.c中断源文件: 新增的DMA中断，剪切到internal_drivers下对应外设源文件中；新增的外设中断，剪切到对应外设设备驱动源文件中（如需要，创建外设驱动文件drv_xxx.c和头文件drv_xxx.h）

- 按照**本工程**board文件夹内容对应替换
- 根据需要，修改CMakeLists_template.txt和CMakeLists.txt文件，如添加文件路径、编译选项等

3. 其他

1. 添加组件：放在components文件夹下，如xxx组件，则在components/xxx下放置组件相关文件，并在CMakeLists.txt中添加xxx组件的编译路径
2. 初始化方法：片上外设若不需要设备驱动，如GPIO、RTC、CRC、DMA等，直接在main函数中进行初始化即可（保持FreeRTOS原初始化方法）；片上外设若需要设备驱动，如SPI、I2C、UART等，则在drivers/internal_drivers文件夹下创建相应的驱动文件，并在CMakeLists.txt中添加驱动文件路径，使用宏INIT_XXX_EXPORT进行初始化（见init_call.h，需要包含该头文件），尽量实现高内聚低耦合，便于进行功能裁减
3. console命令导出方法：本工程重写了shell组件，使用MSH_CMD_EXPORT_ALIAS等宏命令进行导出，使用该宏命令时需要包含shell.h头文件
4. 使用**本工程**时，针对业务变更等应用需求（因单板复用肯定存在），需要使用STM32CubeMX工具重新配置硬件，故而在开发过程中务必保证硬件参数配置与xxx.ioc文件一致，以免造成非预期的更改
5. 在**本工程 使用说明/工程修改说明**路径下，添加了3个demo文档，如下表格，功能开发过程中参考

序号	说明
demo1	新增片上外设，且无需编写设备驱动
demo2	新增片上外设，且需要编写设备驱动
demo3	片上外设功能修改，且需要编写设备驱动

为便于硬件快速复用和验证，使用该工程时，可参考文档《ARM CORE固件模块复用使用指南 V1.1.pdf》第六章，直接通过Cubemx调用脚本的方式配置工程。